FPGA-Accelerated CNN Inference

Revised Master Plan — Version 3.0

As-Built Edition · Incorporates All Implementation Changes

Target: Digilent Nexys 4 (Artix-7 XC7A100T) · Pi 5 Host · Pure VHDL RTL

Revision	3.0 — As-Built
Previous Revision	2.0 — Design Phase
Status	COMPLETED — Fully functional, benchmarked
Classification	Cat / Dog / Human on 32×32 Grayscale Images
SPI Clock	8 MHz (conservative; tested and verified)
Inference Parity	100 / 100 Bit-Exact Match with Golden Model

About This Revision: Version 3.0 is the as-built master plan generated after full project completion. It documents every architectural decision that diverged from the v2.0 design plan — including the merged Capture+Inference pipeline, 16-bit SPI word width, synchronous FIFO replacement, OC_PARALLEL=1, Dense2 output width, custom weight_rom architecture, and the Dummy CS hardware fix. All changes are annotated with rationale.

Table of Contents

1.	Project Goals & Success Criteria (As-Achieved)
2.	Hardware Platform Reference
3.	System Architecture Overview (As-Built)
4.	Fixed-Point Arithmetic Specification
5.	Cycle-Accurate Latency Budget (Actual)
6.	Weight Storage Strategy (As-Implemented)
7.	Communication Protocol (As-Implemented)
8.	VHDL Module Specifications (As-Built)
9.	Timing Constraints & Synthesis Results
10.	Software Tools & File Manifest
11.	Implementation — Changes from v2.0 Plan
12.	Testing & Verification Results
13.	Benchmark & Performance Results
14.	Risk Register — Actual Outcomes

Legend:

CHANGED	Decision changed from v2.0 plan during implementation
NEW	New element not present in v2.0 plan

1. Project Goals & Success Criteria — As Achieved

All seven success criteria defined in v2.0 have been met. The system successfully performs Cat / Dog / Human classification on 32×32 grayscale images and communicates with the Raspberry Pi 5 host over SPI. Classification accuracy is bit-exactly identical to the Q8.8 Golden Model on all 100 tested images.

#	Criterion	Actual Result	Status
SC-1	Accuracy matches golden model bit-exactly	100 / 100 matches (100.0%)	MET
SC-2	FPGA inference-only latency measured	~0.350 ms measured avg; ~0.002 ms true (pipeline flush)	MET
SC-3	FPGA total round-trip latency measured	~3.790 ms (SPI dominates)	MET
SC-4	Pi 5 software inference latency measured	~0.220 ms (TFLite FP32)	MET
SC-5	Speed-up ratio clearly stated	15x fewer clock cycles (FPGA 35K vs Pi 528K cycles)	MET
SC-6	Timing closure at 100 MHz	Zero WNS violations (Vivado verified)	MET
SC-7	Resource utilisation within limits	LUT, BRAM, DSP all within 90% budget	MET

Note on SC-5: The benchmark compares FPGA and Pi compute at the clock-cycle level, not wall-clock time. The system is I/O bound — 69% of total latency is SPI transfer.

2. Hardware Platform Reference

The hardware platform is unchanged from v2.0. The Digilent Nexys 4 (Artix-7 XC7A100T) was used as planned. The MMCM/PLL (Clocking Wizard) was not instantiated in the final implementation — the 100 MHz oscillator drives the design directly via the clk_100mhz port. Timing closure was still achieved.

Resource	Available	Actual Usage	Headroom
Logic Cells	101,440	~35,000	~65%
DSP48E1 Slices	240	~54	~78% headroom
Block RAM (36Kb tiles)	135	~11	~92% headroom
Distributed RAM	Abundant	Line buffers (small depths)	OK
Clock	100 MHz	100 MHz direct (no PLL)	Timing met

Actual Pin Assignments (PMOD)

Signal	Pi 5 Pin	FPGA Pin (PMOD)	Notes
MOSI	SPI0 MOSI (GPIO 10)	JA[1]	Pi to FPGA
MISO	SPI0 MISO (GPIO 9)	JA[2]	FPGA to Pi
SCLK	SPI0 SCLK (GPIO 11)	JA[3]	Pi drives
CS (HW CE0)	SPI0 CE0 (GPIO 8)	JA[4]	Hardware SPI CS (kept as input)
DUMMY CS	GPIO 16 (software)	N/A PMOD	Software-controlled CS — see Section 7
DATA_READY	GPIO 25	JB[1]	Changed: was GPIO 24 in v2.0
GND	GND	GND	Common ground required

DATA_READY GPIO changed from 24 (v2.0) to 25 (v3.0 as-built). Dummy CS on GPIO 16 is a new hardware workaround — see Section 7.

3. System Architecture Overview — As-Built

Key Architectural Departure from v2.0

The most significant architectural change is the merger of the CAPTURE and INFERENCE FSM states into a single concurrent CAPTURE_AND_INFERENCE state. In v2.0, the design plan envisioned buffering all 1156 pixels first, then triggering inference. The as-built design streams pixels directly into the CNN pipeline as they arrive over SPI, exploiting the fact that the CNN is a streaming pipeline that can begin computing before the last pixel is received. This eliminated the need for a large pixel buffer and reduced latency.

As-Built System Hierarchy

```
Pi 5 ■■(SPI 8 MHz, GPIO 16 Dummy CS, GPIO 25 DATA_READY)■■ Nexys 4 FPGA

system_top.vhd (Communication + Master FSM)
■■■ spi_controller.vhd (16-bit Rx/Tx, internal 2-FF sync)
■■■ sync_fifo.vhd (RX: 2048-deep; TX: 16-deep) [was async_fifo]
■■■ cnn_datapath_top.vhd (AXI-Stream pipeline wrapper)
■ ■■■ conv_layer CONV1 (IC=1, OC=16, OC_PAR=1)
■ ■■■ pool_layer POOL1
■ ■■■ padder PAD1
■ ■■■ conv_layer CONV2 (IC=16, OC=32, OC_PAR=1)
■ ■■■ pool_layer POOL2
■ ■■■ padder PAD2
■ ■■■ conv_layer CONV3 (IC=32, OC=32, OC_PAR=1)
■ ■■■ pool_layer POOL3
■ ■■■ flatten.vhd
■ ■■■ dense_engine DENSE1 (512→32, 8 PAR MACs, ReLU)
■ ■■■ dense_engine DENSE2 (32→8, 8 PAR MACs, no ReLU)
■■■ sync_fifo.vhd (TX output buffer)
```

Architecture Differences Summary

Item	v2.0 Plan	v3.0 As-Built	Status
Top-level split	Single top_module.vhd	Split into system_top.vhd + cnn_datapath_top.vhd	CHANGED
CAPTURE + INFERENCE	Separate states; buffer then infer	Single CAPTURE_AND_INFERENCE state; streaming concurrent	CHANGED
FIFO type	Async dual-clock FIFO (Gray-code pointers)	Synchronous FIFO (single 100 MHz clock domain)	CHANGED
SPI data width	8-bit bytes	16-bit words (Q8.8 pixels sent as full 16-bit SPI words)	CHANGED
No MMCM/PLL	Clocking Wizard IP required	Direct 100 MHz oscillator — timing still met	CHANGED
Hardware LED Debugger	Not planned	15 LEDs: FSM state (4 bits), RX empty flag, pixel count (11 bits)	NEW
AXI-Stream interface	Custom valid/ready signals described informally	Full AXI-Stream (in_data/in_valid/in_ready + out_data/out_valid/out_ready)	NEW

4. Fixed-Point Arithmetic Specification

Arithmetic specification is unchanged from v2.0. All weights, biases, and activations use Q8.8 signed fixed-point (16-bit). Accumulators are 48-bit. Bias addition scales to Q16.16 before shifting. ReLU is applied after saturation for all layers except Dense2.

Format: Q8.8 (16-bit signed)

Property	Value / Rule
Range	-128.0 to +127.99609375
Step	1/256 ≈ 0.00390625
Multiply	32-bit intermediate (Q16.16)
Accumulate	48-bit (maps to DSP48E1 P register)
Shift	Right-shift 8 bits after all inputs summed
Bias scale	bias_q8_8 << 8 before addition (brings to Q16.16 domain)
ReLU	Clamp to 0 after saturation — all layers except Dense2
Quantise	round(x_float * 256), clipped to [-32768, 32767]

Conv layer pipeline: 9 parallel multipliers feed a 3-level binary adder tree (sum_A + sum_B + sum_C = mac_sum_reg) before accumulation. This is a 5-stage pipeline: address issue → wait BRAM → multiply → partial sums → accumulate.

5. Cycle-Accurate Latency Budget — Actual Implementation

OC_PARALLEL was set to 1 (not the recommended 2) for all three conv layers. This corresponds to the 'Fully Serial' baseline in v2.0. The estimated total inference latency is therefore ~2.3 ms. In practice, inference is measured to complete in ~0.350 ms on average due to the concurrent streaming pipeline: the FPGA begins computing during SPI reception, so only the pipeline-flush time remains after the last pixel arrives.

Layer	Cycles (est.)	Time @ 100 MHz (est.)
Conv1 + ReLU	~16,454	~165
MaxPool1	~4,128	~41
Padder1	~5,220	~52
Conv2 + ReLU	~131,110	~1,311 (bottleneck, OC_PAR=1)
MaxPool2	~1,040	~10
Padder2	~1,300	~13
Conv3 + ReLU	~65,570	~656
MaxPool3	~260	~3
Flatten	~512	~5
Dense1 + ReLU	~2,560	~26
Dense2	~64	<1
TOTAL	~228,218	~2,283 µs (OC_PAR=1 serial)

Actual measured FPGA 'pure math' time: ~0.350 ms average (OS jitter makes this measurement unreliable — the true flush time is ~0.002 ms). The streaming concurrent pipeline is the reason measured time is much less than the 2.3 ms sequential estimate.

Latency Breakdown — Measured (100 images)

Source	Time	Share	Notes
SPI Transfer (TX)	~2.620 ms	69% of total	8 MHz × 2312 bytes (16-bit pixels)
Linux OS Interrupt Delay	~0.800 ms	21% of total	Kernel scheduler jitter — not FPGA
FPGA Pure Math (meas.)	~0.350 ms	9% of total	Inflated by OS jitter; true ~0.002 ms
SPI Receive (RX)	~0.020 ms	<1% of total	6 result bytes
TOTAL	~3.790 ms	100%	System-level latency

6. Weight Storage Strategy — As Implemented

The v2.0 plan specified Xilinx Block Memory Generator IP with .coe coefficient files. The as-built design uses a custom weight_rom.vhd entity with an impure VHDL initialisation function that reads .mem hex files at elaboration time. For conv layers, the ROM word width is 144 bits (9 × 16-bit weights packed per address, one address per filter-channel pair), allowing all 9 kernel weights to be read in a single BRAM cycle. The ram_style='block' attribute is set to force BRAM mapping. Biases use a 16-bit-wide ROM.

Weight Storage Differences

Item	v2.0 Plan	v3.0 As-Built	Status
Weight ROM implementation	Xilinx Block Memory Generator IP + .coe files	Custom weight_rom.vhd with impure init function reading .mem files	**CHANGED**
Conv weight word width	16-bit per address (one weight)	144-bit per address (9 × 16-bit = full 3×3 kernel in one read)	**CHANGED**
Bias storage	16-bit-wide BRAM or LUTRAM	weight_rom.vhd with WORD_WIDTH=16 (same entity, different generic)	**CHANGED**
Dense weight organization	8 separate narrow BRAMs or wide BRAM	Single weight_rom with WORD_WIDTH = PARALLEL_MACS × 16 (128-bit)	**CHANGED**
Dense biases	BRAM-based	Impure function directly in dense_engine.vhd (inline init_bias)	**CHANGED**

Memory Sizing (As-Built)

Layer	Parameters	ROM Config	BRAM Usage
Conv1 weights	3×3×1×16 = 144	16 entries × 144-bit	1 BRAM tile (with ram_style=block)
Conv1 biases	16	16 entries × 16-bit	LUTRAM
Conv2 weights	3×3×16×32 = 4,608	512 entries × 144-bit	~1.5 tiles
Conv2 biases	32	32 entries × 16-bit	LUTRAM
Conv3 weights	3×3×32×32 = 9,216	1,024 entries × 144-bit	~3 tiles
Conv3 biases	32	32 entries × 16-bit	LUTRAM
Dense1 weights	512×32 = 16,384	2,048 entries × 128-bit	~5.5 tiles
Dense1 biases	32	inline array	Registers
Dense2 weights	32×8 = 256	512 entries × 128-bit	< 1 tile
Dense2 biases	8	inline array	Registers
TOTAL	~30K 16-bit words	~11 BRAM tiles	~8% of Nexys 4

7. Communication Protocol — As Implemented

7.1 Dummy CS Hardware Fix

The plan assumed the hardware SPI CS (CE0, GPIO 8) would control the SPI transaction. In practice, a Dummy CS workaround was required: a separate GPIO pin (GPIO 16) is driven by software in spi.py as a software-controlled CS. The native CE0 pin is still wired to JA[4] for the FPGA to detect transaction boundaries, but the spidev xfer2() calls are framed by manually toggling GPIO 16 (cs_n.off() / cs_n.on()).

7.2 SPI Speed

v2.0 planned 12 MHz SPI. The as-built design operates at 8 MHz. The plan already recommended starting at 8 MHz and increasing; 8 MHz proved reliable on jumper wires and was kept as the final value.

7.3 SPI Word Width Change

v2.0 described 8-bit byte transfers. The as-built design uses 16-bit SPI words (G_TX_WIDTH=16, G_RX_WIDTH=16). Each 32×32 pixel, after zero-padding to 34×34 and Q8.8 conversion, is transmitted as a single 16-bit SPI word. This means the image payload is 1,156 16-bit words = 2,312 bytes on the wire (not 1,156 bytes as in v2.0).

7.4 DATA_READY Protocol

The revised single-assert DATA_READY protocol from v2.0 Section 7.1 was retained, with the addition of PRE_LOAD and WAIT_FOR_SAFE_LOAD states to synchronise BRAM read latency with the SPI transmitter. DATA_READY is held high throughout SEND_RESULTS and WAIT_FOR_SAFE_LOAD, de-asserting only when CS returns high (IDLE).

Protocol Differences

Item	v2.0 Plan	v3.0 As-Built	Status
SPI clock	12 MHz (test at 8 MHz first)	8 MHz (final — reliable on jumper wires)	CHANGED
SPI word width	8-bit bytes (1,156 bytes image)	16-bit words (2,312 bytes; 1,156 Q8.8 pixels)	CHANGED
DATA_READY GPIO	GPIO 24	GPIO 25	CHANGED
CS control	Hardware SPI CE0 only	Hardware CE0 input + software Dummy CS GPIO 16	CHANGED
SEND FSM states	SEND_RESULTS only	PRE_LOAD → SEND_RESULTS → WAIT_FOR_SAFE_LOAD	CHANGED
Result bytes	6 bytes (3 × Q8.8 logits, 2 bytes each)	6 bytes (unchanged)	SAME
Byte order	MSB-first, big-endian logit[0..2]	Identical	SAME

8. VHDL Module Specifications — As-Built

All VHDL modules use a consistent AXI-Stream interface: in_data / in_valid / in_ready (slave) and out_data / out_valid / out_ready (master). Reset is active-high synchronous on the 100 MHz clock.

File	Type	Notes vs v2.0
cnn_config.vhd	Package	Global constants. SAME as v2.0 except: DATA_W renamed DATA_WIDTH; SPI_FIFO_DEPTH removed; SPI_DATA_WIDTH=16 added; IMG_BYTES renamed IMG_SIZE.
system_top.vhd	Top	NEW in v3.0 as a separate file. Contains master FSM with states: IDLE, CAPTURE_AND_INFERENCE, PRE_LOAD, SEND_RESULTS, WAIT_FOR_SAFE_LOAD. Also contains hardware LED debugger, 2-FF synchronisers for SCLK/CS, and combined_reset (sys_reset OR soft_reset) logic.
cnn_datapath_top.vhd	Top (CNN)	NEW wrapper. Instantiates all CNN stages with generics for weight file paths. AXI-Stream in/out. Dense2 OUT_SIZE=8 (not 3 as planned — see Section 8 note).
spi_controller.vhd	Module	Reused with change: G_TX_WIDTH / G_RX_WIDTH generics now default to 16 (was 8). Internal 2-FF synchronisers for CS and SCLK.
sync_fifo.vhd	Module	REPLACES async_fifo.vhd. Single-clock synchronous FIFO. RX instance: ADDR_WIDTH=11 (2,048 entries). TX instance: ADDR_WIDTH=4 (16 entries).
conv_layer.vhd	Module	CHANGED internals. 5-stage pipeline: address issue, BRAM latency, 9-parallel multiply, binary adder tree (sum_A+B+C), accumulate. OC_PARALLEL=1 for all instances. Weight ROM uses 144-bit wide addressing.
weight_rom.vhd	Module	NEW custom BRAM ROM. Generic WORD_WIDTH and DEPTH. Impure init function reads .mem file at elaboration. ram_style=block attribute set.
pool_layer.vhd	Module	Unchanged from plan. 2-stage pipeline with stall control.
padder.vhd	Module	Unchanged from plan. 3-state FSM: PAD_TOP_ROW, COPY_ROWS, PAD_BOT_ROW.
flatten.vhd	Module	Unchanged from plan. Pass-through stream, element counter.
dense_engine.vhd	Module	CHANGED states: IDLE→LOAD_BUFFER→COMPUTE_WAIT→COMPUTE→FLUSH_MAC→DRAIN_OUT. Dedicated BRAM process for input buffer. Inline impure init_bias function for bias array.

Dense2 OUT_SIZE=8 note: The Dense2 instance uses OUT_SIZE=8 with all 8 PARALLEL_MACs, outputting 8 values. Only the first 3 represent valid class logits (Cat, Dog, Human). The remaining 5 outputs are unused. This was an implementation convenience — the dense_engine works in fixed PARALLEL_MACS increments.

9. Timing Constraints & Synthesis Results

Design closes timing at 100 MHz with zero WNS violations, as verified in Vivado's timing summary report. The likely critical path is through the 5-stage conv layer DSP48 accumulator chain; pipeline registers were added where needed.

Key XDC Constraints (As-Used)

Primary clock (100 MHz oscillator, pin E3), PMOD JA SPI pins, PMOD JB DATA_READY, and 16-bit LED port. All SPI signals set as asynchronous via set_clock_groups (SCLK treated as async input with 2-FF synchroniser in RTL). No MMCM/PLL clock constraint needed (direct oscillator).

Verified Resource Utilisation (from Vivado)

Full utilisation summary visible in the project's utililization summary.png report. All categories (LUT, BRAM, DSP) are below the 90% budget. BRAM usage is particularly low (~11 tiles of 135 available = ~8%).

10. Software Tools & File Manifest

Python Scripts (As-Built)

File	Purpose	Phase	Status vs v2.0
train_cnn.py	Train Keras model, save tiny_ai_brain.h5 and .tflite	Phase 0	SAME
export_weights.py	Quantise to Q8.8, generate .npy and .mem files	Phase 0	SAME
golden_model.py	Bit-exact Q8.8 integer inference (Python int)	Phase 1+	SAME
generate_testbench_data.py	Hex files from golden model for VHDL testbenches	Phase 1	SAME
spi.py	FPGACNN class: preprocess → Dummy CS → SPI → logits + timing	Phase 4/5	CHANGED
benchmark.py	TFLite vs FPGA vs Golden Model; OS calibration loopback	Phase 5	CHANGED
spi_test.py	SPI loopback test (pre-existing)	Phase 4	SAME

VHDL Sources (As-Built)

File	Type	Notes
cnn_config.vhd	Package	Minor constant renames
spi_controller.vhd	Module	16-bit width generics
spi_receiver.vhd	Module	Reused unchanged
spi_transmitter.vhd	Module	Reused unchanged
sync_fifo.vhd	Module	NEW — replaces async_fifo
weight_rom.vhd	Module	NEW — custom BRAM ROM
conv_layer.vhd	Module	CHANGED internals
pool_layer.vhd	Module	Unchanged
padder.vhd	Module	Unchanged
flatten.vhd	Module	Unchanged
dense_engine.vhd	Module	CHANGED FSM and init
cnn_datapath_top.vhd	Top (CNN)	NEW wrapper entity
system_top.vhd	Top	REPLACES top_module.vhd
nexys4_top.xdc	Constraints	As-built pin assignments

11. Implementation — Summary of All Changes from v2.0

The table below consolidates every change made during implementation compared to the v2.0 design plan, with rationale.

Item	v2.0 Plan	v3.0 As-Built	Status
CAPTURE + INFERENCE merged	Separate FSM states; buffer pixels then infer	Single state; stream pixels directly into CNN during SPI reception	CHANGED
Async → Sync FIFO	Dual-clock async FIFO with Gray-code pointers for CDC	Synchronous FIFO (single clock domain); SPI controller handles its own sync	CHANGED
SPI word width	8-bit (bytes)	16-bit (Q8.8 pixels as full words); image payload doubles to 2312 bytes	CHANGED
SPI speed	12 MHz (target)	8 MHz (final verified value; stayed at conservative test speed)	CHANGED
OC_PARALLEL	2 (recommended)	1 (all conv layers); streaming pipeline compensates in practice	CHANGED
Dense2 output size	32 → 3	32 → 8 (only 3 used; padded to fill PARALLEL_MACS=8)	CHANGED
Weight ROM	Xilinx Block Memory Generator IP + .coe files	Custom weight_rom.vhd + .mem files; 144-bit wide for conv layers	CHANGED
MMCM/PLL	Required — Clocking Wizard IP	Not used; direct 100 MHz oscillator (timing still met)	CHANGED
DATA_READY GPIO	GPIO 24	GPIO 25	CHANGED
Dummy CS pin	Not planned	GPIO 16 software-controlled CS workaround	NEW
FSM ERROR state	Explicit ERROR state with watchdog timer	Emergency abort on early CS deassertion; no watchdog counter	CHANGED
Hardware LED Debugger	3 status LEDs (idle, inference, error)	16-bit LED bus: FSM state + RX empty flag + pixel count	NEW
cnn_datapath_top.vhd	Monolithic top_module.vhd	Separate cnn_datapath_top.vhd wrapper for AXI-Stream CNN pipeline	NEW
Conv layer internals	Streaming pipeline with simultaneous OC_PARALLEL output channels	5-stage pipeline: addr, BRAM latency, 9×multiply, binary tree, accumulate	CHANGED
Benchmark method	Python Q8.8 integer inference vs FPGA inference-only	TFLite FP32 + Golden Model Q8.8 + dynamic OS calibration loopback	CHANGED

12. Testing & Verification Results

The verification pyramid from v2.0 was followed: golden model → module unit tests → chain simulation → system simulation → hardware-in-loop. The golden model was the authoritative reference; all VHDL outputs were compared bit-for-bit.

Verification Pyramid Outcomes

Level	Test	Result	Notes
Level 0	Golden Model (Python integer Q8.8)	PASS	Bit-exact reference established for all layers
Level 1	Module unit tests (each VHDL module)	PASS	All modules match golden model bit-for-bit
Level 2	CNN chain simulation	PASS	End-to-end logits match on 3+ test images
Level 3	System simulation (SPI + FSM + CNN)	PASS	Full transaction verified in Vivado Simulator
Level 4	Hardware-in-loop (board, real images)	PASS	100/100 matches on randomised test set

Functional Correctness (Hardware vs Golden Model)

100 randomised test images (Cat, Dog, Human) were classified. The FPGA produced identical results to the Q8.8 Golden Model on all 100. Occasional disagreements between FPGA and TFLite are expected and are due to quantisation loss (FP32 vs Q8.8), not a hardware defect.

13. Benchmark & Performance Results

Benchmark methodology: FPGA and TFLite (Pi 5) are run on the same 100 randomised images. OS interrupt overhead is dynamically calibrated per image using a GPIO loopback (GPIO 17 → GPIO 27). The Pi's compute advantage in wall-clock time is explained entirely by the I/O bottleneck, not by compute architecture.

Metric	Value	Notes
FPGA inference only (est. serial)	~2,283 µs	Sequential worst-case (OC_PAR=1)
FPGA pipeline flush (true math)	~2 µs	After streaming concurrent capture
FPGA pure math (measured avg)	~350 µs	Inflated by OS scheduler jitter
SPI TX transfer (image)	~2,620 µs	2,312 bytes @ 8 MHz
SPI RX (results)	~20 µs	6 bytes @ 8 MHz
Linux OS interrupt delay	~800 µs	Measured via GPIO loopback calibration
FPGA total system latency	~3,790 µs	SPI-bound; 69% is I/O
Pi 5 TFLite FP32 inference	~220 µs	FP32 model, warmed CPU cache
Clock cycles — Pi 5 (2.4 GHz)	~528,000	For 0.220 ms inference
Clock cycles — FPGA (100 MHz)	~35,000	Conservatively from 0.350 ms
Compute efficiency advantage	~15×	FPGA uses 15× fewer cycles

Benchmark Conclusion Statement

FPGA inference-only clock cycles: ~35,000 @ 100 MHz. Pi 5 TFLite inference clock cycles: ~528,000 @ 2.4 GHz. Compute efficiency: FPGA uses ~15× fewer clock cycles. FPGA total round-trip (including SPI): ~3.79 ms — I/O bound (69% SPI). The system is I/O bound, not compute bound. On a Zynq SoC with direct AXI memory access, the I/O bottleneck would be eliminated and the FPGA's compute advantage would dominate.

14. Risk Register — Actual Outcomes

Risk	Likelihood	Impact	Planned Mitigation	Actual Outcome
Conv2/Conv3 timing closure at 100 MHz	Medium	High	Pipeline registers added where needed	DID NOT OCCUR — timing closed cleanly at 100 MHz
BRAM IP initialisation wrong	Medium	Medium	Xilinx IP + .coe format careful verification	AVOIDED — switched to custom weight_rom.vhd + .mem; no IP dependency
SPI signal integrity at 12 MHz	Medium	Medium	Test at 8 MHz first; use short wires	PARTIALLY OCCURRED — stayed at 8 MHz; 12 MHz not tested on board
Pi OS jitter disrupts DATA_READY polling	Low	Low	Hold DATA_READY protocol robust to jitter	OCCURRED — OS jitter significantly affects measured FPGA time; mitigated by GPIO loopback calibration in benchmark.py
Golden model / VHDL mismatch	High	High	Build golden model first; verify every op	DID NOT OCCUR — 100/100 bit-exact match achieved
Dataset accuracy too low	Medium	Medium	Use augmentation; target >=1500 images/class	DID NOT OCCUR — model accuracy acceptable
FPGA slower than Pi (inference only)	Low	Medium	Optimise OC_PARALLEL; compare to Python not C	PARTIALLY — FPGA 15× more clock-cycle-efficient; total system latency slower due to SPI I/O bottleneck
Hardware CS pin issue	N/A (not in v2.0)	N/A	Not anticipated	NEW — Required Dummy CS GPIO 16 workaround

End of FPGA-Accelerated CNN Inference Master Plan v3.0 — As-Built Edition. All phases completed. All success criteria met.